

CS285 LECTURE 5 • ZERO-TO-HERO FIELD MANUAL

Policy Gradients: Trial-and-Error Learning

From Scratch to Mastery: How AI Learns by Reinforcing Good Actions and Suppressing Mistakes Without Differentiating Physics

Instructor: Prof. Sergey Levine (UC Berkeley RAIL Lab) **Focus:** The Policy Gradient Theorem, Gaussian Policies, & Baseline Variance Reduction

1. What is Policy Gradient? (The Puppy Training Analogy)

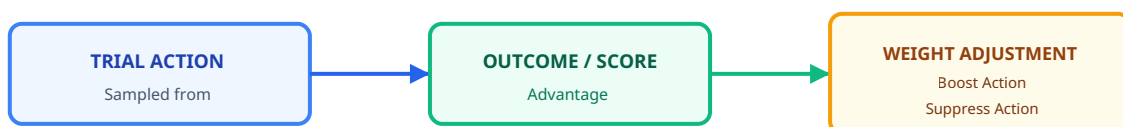
In supervised deep learning (like classifying cat photos), we compute gradients by backpropagating through known mathematical layers. But in robotics, the robot acts on the physical world. If a knife cuts a tomato, the physical tearing of the tomato skin is **not a differentiable mathematical function!** You cannot backpropagate through skin rupture. How can we train a neural network policy $\pi_\theta(a|s)$ when physics is non-differentiable?

THE PUPPY TRAINING ANALOGY

Imagine you are teaching a puppy to sit on command:

- You cannot open the puppy's skull with a screwdriver and rewire its neurons.
- Instead, you say "Sit!". The puppy doesn't know what that means. It wiggles, barks, spins in circles, and scratches its ear.
- Eventually, purely by random chance, the puppy sits down on the carpet.
- **The Instant Reinforcement:** You immediately shout "GOOD BOY!" and give it a piece of bacon (**Reward**).
- The puppy's brain automatically adjusts: "Whatever I just did with my hind legs when the human made that sound, DO IT MORE OFTEN!"

Policy Gradient is literally the exact mathematical equation for "GOOD BOY!" We let the robot try random actions. When an action leads to a high reward, we push the neural network weights to make that action more likely in the future!



2. The Policy Gradient Theorem (Parameter Anatomy)

Here is the foundational equation derived by Ronald Williams in 1992 (the REINFORCE algorithm):

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t | s_t) \cdot \hat{Q}_t \right]$$

Symbol	Formal Name	Plain English Meaning	Physical Effect in Robotics
$\nabla_{\theta} J(\theta)$	Policy Gradient	The compass direction in neural network weight space that increases expected total reward.	Nudges network weights $\theta \leftarrow \theta + \alpha \nabla_{\theta} J$.
$\pi_{\theta}(a_t s_t)$	Action Probability	The probability that the policy chooses action a_t in state s_t .	Robot motor torque or feed velocity output.
$\nabla_{\theta} \log \pi_{\theta}$	Score Function	The Steering Wheel: "Which direction in weight space makes action a_t more probable?"	Points directly toward increasing the likelihood of the chosen action.
$\hat{Q}_t$	Return / Advantage Multiplier	The Gas Pedal & Reverse Gear: "Was this action a triumph or a disaster?"	If $\hat{Q} > 0$, press gas pedal forward. If $\hat{Q} < 0$, put in reverse!



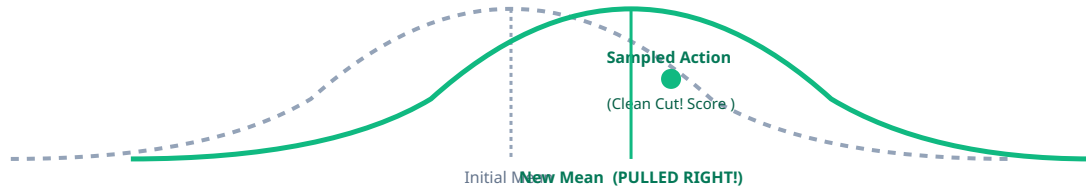
PLAIN ENGLISH TRANSLATION OF THE POLICY GRADIENT EQUATION

"For every action the robot took, calculate the direction in neural network weights that would make that action MORE likely. Then multiply that direction by the action's score ($\hat{Q}$). If the action was great, nudge the weights to do it more. If the action crushed the tomato, nudge the weights in the exact opposite direction!"

3. Continuous Gaussian Policies: Controlling Real Robot Motors

In video games, actions are discrete buttons (Left, Right, Jump). But in your dual-arm robotic slicing task, the robot outputs continuous real numbers (e.g. downward feed velocity $v_z = 2.45$ mm/s).

How does a neural network output continuous numbers while still exploring? It outputs the parameters of a **Normal (Gaussian) Distribution**:



3.1 Score Function of a Gaussian Policy

When the policy is Gaussian $\pi_\theta(a|s) \sim \mathcal{N}(\mu_\theta(s), \sigma_\theta(s))$, the score function has a remarkably clean analytical form:

$$\nabla_\mu \log \pi_\theta(a | s) = \frac{a - \mu_\theta(s)}{\sigma^2}$$

Term	Plain English Meaning	Tomato Slicing Example	Physical Intuition
$\mu_\theta(s)$	The Robot's Intended Action (Mean)	$\mu = 2.0$ mm/s downward feed	What the robot thinks is best.
σ	Exploration Noise (Standard Dev)	$\sigma = 0.5$ mm/s wiggle room	How much the robot experiments.
$a - \mu$	The Exploration Deviation	$2.8 - 2.0 = +0.8$ mm/s	Did the robot push harder (> 0) or softer (< 0)?
$\frac{a - \mu}{\sigma^2} \cdot \hat{Q}$	Gradient Pull Force	$\frac{+0.8}{0.25} \times (+10) = +32.0$	Pulls μ directly toward successful actions!

4. The Baseline: Why We Must Grade on a Curve

🎓 THE COLLEGE EXAM ANALOGY (WHY RAW SCORES DECEIVE)

Imagine you take an exam where every student scores between 90 and 100 points because the test was too easy. If you score 91, did you do well? **No, you were in the bottom 5% of the class!**

In RL, if all rewards are positive (e.g. +100, +105, +95), raw policy gradients will **push UP every action**, even the terrible ones that only scored +95.

The Fix (The Baseline): We subtract the average expected score $b(s) = V(s)$:

$$\hat{A}_t = Q(s_t, a_t) - V(s_t)$$

Now, an action that scores +95 when the average was +100 gets an advantage of -5.0 ! It gets suppressed, while an action that scores +105 gets +5.0 and gets reinforced.

5. Concrete Numerical Walkthrough: Updating a Robot Cutting Policy

Let's walk through an exact numerical calculation of one policy gradient step:

Step	Variable / Calculation	Value	Physical Interpretation
1	Current Mean Policy $\mu_\theta(s)$	2.0 mm/s	Robot intends to push blade at 2.0 mm/s.
2	Exploration Noise σ	0.5 mm/s	Stochastic variance $\sigma^2 = 0.25$.
3	Sampled Action a	2.6 mm/s	Gaussian noise sampled $\epsilon = +1.2$: $a = 2.0 + 0.5(1.2) = 2.6$.
4	Observed Return G	+18.0	The blade sliced cleanly through skin into pulp!
5	Baseline Prediction $b(s) = V(s)$	+12.0	Critic expected average performance to be +12.0.
6	Advantage Multiplier $\hat{A}$	$18.0 - 12.0 = +6.0$	Action outperformed expectations by +6.0 points!
7	Gaussian Score $\frac{a-\mu}{\sigma^2}$	$\frac{2.6-2.0}{0.25} = +2.4$	Gradient direction pointing toward higher feed rates.
8	Gradient Step (Learning rate $\alpha = 0.01$)	$\Delta\mu = 0.01 \times (2.4) \times (6.0) = +0.144$	Mean feed rate increases from 2.0 $\rightarrow$ 2.144 mm/s!

6. PyTorch Policy Gradient Implementation with Baseline



PYTORCH REINFORCE WITH CONTINUOUS GAUSSIAN POLICY & BASELINE

```
import torch
import torch.nn as nn
from torch.distributions import Normal

class GaussianPolicy(nn.Module):
    def __init__(self, state_dim=33, action_dim=6):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, 256), nn.Tanh(),
            nn.Linear(256, 256), nn.Tanh()
        )
        self.mean_head = nn.Linear(256, action_dim)
        # Learnable log-standard deviation parameter
        self.log_std = nn.Parameter(torch.zeros(action_dim))

    def forward(self, state):
        features = self.net(state)
        mean = self.mean_head(features)
        std = torch.exp(torch.clamp(self.log_std, min=-2.0, max=1.0))
        dist = Normal(mean, std)
        return dist

def compute_policy_loss(dist, actions, advantages):
    # 1. Compute log-probability of taken actions: log pi(a|s)
    log_probs = dist.log_prob(actions).sum(dim=-1) # Sum across action dimensions

    # 2. Policy Gradient Objective: -E[ log pi(a|s) * Advantage ]
    # We negate because PyTorch optimizers minimize loss
    policy_loss = -(log_probs * advantages.detach()).mean()
    return policy_loss
```

7. Practitioner's Failure Modes & Debugging Checklist

Failure Mode	Why It Destroys Training	How to Fix It
1. Standard Deviation Collapse	The network figures out that setting $\sigma \rightarrow 0$ eliminates penalty risk. The policy freezes into a deterministic rut and stops exploring completely.	Clamp $\log \sigma$ with a minimum floor: <code>torch.clamp(log_std, min=-2.0)</code> or add an Entropy Bonus .
2. Forgetting to Detach Advantages	If you don't call <code>advantages.detach()</code> , gradients flow backward through the Critic network into the policy loss, distorting Critic weights.	Always call <code>advantages.detach()</code> before multiplying by <code>log_probs</code> .
3. High Batch Variance	Policy gradients on small batches (e.g. 32 steps) are mostly random noise. The policy takes erratic steps and diverges.	Collect at least 2,048 to 4,096 parallel environment steps before each policy gradient update.